

KDP Coloring-Book Automation — Operation Manual

A practical, start-to-finish guide to producing a coloring book with this project. It ties together the AI team roles, the metadata tracking sheet, the image pipeline, and the pre-upload validation into one repeatable flow.

1. What this does

It turns a book concept into **print-ready files + validated metadata** for Amazon KDP, with a **human-approval gate** before anything is published. The chain of command: **you (NgocETurnal, Project Owner)** → **Gen (Project Manager Agent)** → **Dalton (Team Manager)** → **the three production levels** (Level 1 research & planning, Level 2 production, Level 3 upload staging). The agents draft, generate, check, and assemble — you approve.

2. One-time setup

```
cd kdp-coloring-book-automation
bash scripts/setup.sh
source .venv/bin/activate
pytest validation/test_validate.py -q      # should show all tests passing
```

Gen (the Image Generation Agent) is allowed to run this setup itself to bootstrap a run — so a fresh start is never blocked waiting on setup.

3. The production flow

1. **Pick a concept** and priority (adult sarcastic/gift → kids → wellness).
2. **Log it** in the tracking sheet's `META_DRAFT` tab; note the `task_id`.
3. **Generate art:**

```
python image_pipeline/generate_art.py work/<task>/raw --pages 50 \
    --prompt "your theme, bold clean outlines, no shading" --backend cloud
```

(Use `--backend placeholder` to dry-run with no API/GPU.)

1. **Apply the bleed rule & build PDFs:**

```
python image_pipeline/apply_bleed.py work/<task>/raw work/<task>/split
python image_pipeline/build_interior_pdf.py work/<task>/split/non_bleed \
    work/<task>/interiors/interior_nonbleed.pdf --format print_square_nonbleed
```

1. **Hand off to QA** with a checksum packet:

```
python image_pipeline/write_handoff_packet.py work/<task>/interiors/interior_nonbleed.pdf
    work/<task>/handoff_qa.json --task-id <task> --book-slug <slug> \
    --stage QA --from-team Interior_Creation --to-team QA
```

1. **Validate metadata** before approval:

```
python validation/validate_csv.py <exported META_APPROVED>.csv --final-gate
```

1. **Human approval** — you review and approve. Nothing publishes automatically.
2. **Upload** using only the approved packet.
3. **Post-launch** — log results back to the CHANGELOG/ARCHIVE tabs.

4. Print specs (already enforced by the code)

- Print interior: **8.5" × 8.5"**; bleed pages become **8.625" × 8.75"**; **300 DPI**.
- Ebook: **8.5" × 11"**, non-bleed.
- Non-bleed outer margin $\geq 0.25"$; gutter per KDP's page-count table.

5. Running it alongside ROM on one mini-PC

If this shares the AOOSTAR mini-PC with the ROM offline library, see [../docs/hosting-and-setup.md](#) : keep them on separate drives, run one heavy job at a time, and use a cloud image backend (or an OCuLink eGPU later) since the built-in iGPU is weak for local diffusion.

6. Guardrails

- **No auto-publish** — human approval is mandatory.
- **Checksum every handoff** — don't skip the packet step.
- **META_APPROVED is read-only after approval** — re-approve through a new draft.

7. Troubleshooting

Problem	Try
<code>build_interior_pdf.py</code> says wrong px size	Run <code>apply_bleed.py</code> first, or pick the matching <code>--format</code> .
Cloud backend errors	It's not configured by default — implement <code>_backend_cloud</code> and set <code>IMAGE_API_KEY</code> , or use <code>--backend placeholder</code> .
Validator fails on a good-looking row	Read the <code>ERROR</code> line; rules live in <code>validation/rules.yaml</code> .
Apps Script menu missing	Run <code>onOpen</code> once in the Apps Script editor and authorize.

This project prepares files and metadata for KDP; it does not bypass Amazon's review. You are responsible for content, rights, and compliance with KDP policy.